

UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 6

Microsoft Foundry, First Contact

The enterprise platform, opened for the first time. You have built every piece of it by hand already, so today you will recognise all of it, and see what a company adds on top.

SECTION 00

A different kind of lecture

Today you watch. Next lecture you are back on the keyboard.

RECALL

Five lectures by hand. On purpose.

THE MODEL

A reflex agent. One percept, one action, no memory.

TOOLS

Ordinary functions you exposed. The hands.

THE LOOP

Think, act, observe. The engine, with a budget.

MEMORY

A messages list you resend every turn.

WHY THAT MATTERS TODAY

A platform like Foundry can feel like magic when you first open it. It is not going to feel like magic to you, because you have already written every single one of those five things yourself. Today you are just seeing the industrial version of a machine you have already built in your kitchen.

Today

- What Foundry actually is, and the two names it had before this one.
- The one structural choice that trips everybody: projects, not hubs.
- A tour of the left navigation, mapping each panel to something you already built.
- The playground, and three things it quietly teaches that are worth more than the tour.
- Deployments, deprecation, and the cost cap that is the only thing standing between you and a surprise bill.

HOLD ON TO THIS

You do not need a Foundry account to pass this course. Everything is also on the free GitHub lane. But you should understand this platform, because it is what a large part of the industry actually runs on.

SECTION 01

What it is, and what it is called

A platform with an identity crisis, and one choice that matters.

The name has changed twice in two years

2023

Azure AI Studio

The original name. You will still find tutorials using it.

LATE 2024

Azure AI Foundry

Renamed and re-centred around building agents.

2026

Microsoft Foundry

The current name. Same platform, same keys, same everything.

WHY I AM MAKING YOU LEARN THREE NAMES FOR ONE THING

Because when you search for help, you will land on all three, and you need to know they are the same platform. This is not trivia. It is the single most common source of confusion for anyone starting Foundry in 2026, and now it will not confuse you.

What Foundry actually is

THE LAYMAN VERSION

A workshop with the tools already bolted down

You built your agent on a kitchen table with hand tools. Foundry is the professional workshop: the same work, but the models, the deployments, the monitoring, the safety filters, and the access controls are all already installed and wired together.

THE TECHNICAL VERSION

A managed platform for building, deploying and operating agents

One place to reach a catalogue of models, deploy them behind an endpoint, attach tools and memory, add guardrails, run evaluations, and watch it all in production. The pieces you wrote by hand, offered as managed services.

HOLD ON TO THIS

Managed is the key word. You stop maintaining the plumbing and start paying for someone else to maintain it. That trade is the entire value proposition of every cloud platform, and it is a real trade, not a free win.

The one choice that trips everybody

THE OLD WAY

Hubs

The original structure, built around a shared hub resource. Still there for backwards compatibility. Every older tutorial tells you to create one.

THE CURRENT WAY

Projects

The project first structure everything new is built around. The agent service, evaluations, the current model catalogue, all wired up here first.

THE PRACTICAL RULE

When you open the portal, make sure the New Foundry toggle is on, and create a project. If any guide tells you to create a hub, it was written for the old portal. I make this mistake sound small. It is not. Build on the wrong one and half the features you came for are simply not there.

SECTION 02

The tour

Ten panels down the left. You built most of them already.

The left navigation, and where you have seen it before

Agents

The no code agent builder

Your tiny_agent, with a UI

Fine-tune

Train a model on your data

New. Unit 3 touches it

Tools

Functions agents can call

Your REGISTRY and schema

Memory

What an agent remembers

Your messages list

Evaluations

Score an agent against a standard

Unit 5

Models

Deploy and manage endpoints

Your lanes.py, as a service

Services

Speech, vision, language APIs

More tools for the toolbox

Knowledge

Documents an agent can search

RAG. Unit 5

Data

Datasets and connections

New

Guardrails

Content filters and limits

Your budget and whitelist

The three you have not built yet

KNOWLEDGE

Give an agent a pile of documents and let it search them at answer time. This is retrieval augmented generation, and it is how you stop a model inventing facts. Unit 5.

FINE-TUNE

Adjust the model itself on your own examples, rather than instructing it each time. Powerful, expensive, and the wrong first reach for most problems. We touch it in Unit 3.

EVALUATIONS

Score an agent against a fixed standard, automatically, so you know whether a change made it better or worse. This is how you stop guessing. Unit 5.

HOLD ON TO THIS

Notice two of the three point at Unit 5. That is the testing and reliability unit, and it is the one that separates a demo from a product. These panels exist because industry needs them, which is exactly why we spend real time there.

SECTION 03

The playground

Where you talk to a model, and where it quietly teaches you three things.

The playground, in one look

ON THE LEFT

The controls

Instructions, which is the system prompt. Tools you can switch on. Knowledge and memory you can attach. Everything you wired by hand, now a panel with a toggle.

ON THE RIGHT

The conversation

You type, the model answers, and it shows you the model name, the time taken, and the token count under every message. That last part is the interesting bit.

THE INSTRUCTIONS BOX IS A SYSTEM PROMPT

It comes pre-filled with "You are an AI assistant that helps people find information." That is a system prompt, live and editable, exactly the thing from Lecture 3. Change it in front of the room and watch the behaviour change. The platform did not invent a new concept. It gave your concept a text box.

Lesson one, the model does not know itself

"I am ChatGPT, an AI assistant built by OpenAI..."

The deployment is called chat-demo. The model underneath is gpt-5-mini. It said neither. It guessed, confidently, and cited a web search to back up the guess.

WHY THIS IS A REAL LESSON, NOT A GOTCHA

A model has no reliable knowledge of its own name, version, or deployment. It was trained before it was given that name. So "what model are you" is one of the least reliable questions you can ask, and if your agent ever needs to know, you tell it in the system prompt. You do not ask it to introspect. It will make something up.

ONE SHORT QUESTION COST 9436 TOKENS

Lesson two, the bill is not the message

I asked one short question. The token counter under the answer read 9436. Where did nine thousand tokens come from, for a one line question?

WHAT I SAW

A short question, a huge count

The visible conversation was tiny. The billed token count was not. Those two numbers had almost nothing to do with each other.

WHAT ACTUALLY HAPPENED

The web search tool fired

A tool was switched on. It ran a search, and the entire search result got stuffed into the context for the model to read. You pay for every token of that, even though you never see it.

THE LESSON FROM LECTURE 3, MADE REAL

What you pay for is the whole transcript, including the parts a tool quietly injected. Turn tools on without thinking and your costs move in ways the chat window does not show you. Watch the token counter, not the message length.

Lesson three, turn off what you do not need

IT COSTS EXTRA

Grounding is billed

The web search tool uses a paid grounding service on top of the model. Both of those numbers are yours.

IT LEAVES THE BOUNDARY

Data goes outside Azure

The portal warns you plainly: turn this on and your data flows outside the Azure compliance boundary. For a demo, fine. For real customer data, a decision with consequences.

THE HABIT TO FORM NOW

Every tool you attach is a cost, a dependency, and a piece of attack surface. The default should be off, and you switch a tool on deliberately, knowing what it costs and where the data goes. This is a Unit 6 security principle showing up early, and it is worth forming the habit today.

SECTION 04

Deployments

The part where you choose a model, and the part where you cap the cost.

A deployment is not a model

THE MODEL

gpt-5-mini

The thing that does the reasoning. Sitting in a catalogue, shared, not yet yours.

THE DEPLOYMENT

chat-demo

Your named, running instance of that model, behind an endpoint, with your rate limit and your filter. This is the thing your code actually talks to.

THE MISTAKE THAT COSTS EVERYONE AN HOUR

In your code, MODEL is the deployment name, chat-demo, not the model name, gpt-5-mini. Get this wrong and you get a 404 that reads as though the model does not exist, so you go looking in entirely the wrong place. It is almost always this. Write it on your hand.

DEPRECATED MODELS

The catalogue has a trap in it

Setting up the course account, I went to deploy the obvious cheap model. The portal showed me a banner: deprecated.

WHAT DEPRECATED MEANS

It works today, not for long

A deprecated model still runs, so it is tempting. But it has a retirement date, and when that arrives your endpoint stops answering, mid semester, with no warning in your code.

WHAT I DID INSTEAD

Checked the lifecycle, chose current

I looked at the Lifecycle field, rejected two deprecated models, and picked gpt-5-mini, marked Generally Available. Two minutes of checking against a broken demo in November.

THE SAME LESSON AS BOT FRAMEWORK

Remember from Lecture Zero, the Bot Framework SDK was retired during this course, so we do not build on it. Same principle here. Before you build on anything, check that it is not on its way out. Deprecation is a date, and dates arrive.

The one slider that protects your money

When you deploy, you set a tokens per minute rate limit. I dragged it to the minimum, 10000.

WHAT IT DOES

Caps the spend rate

The deployment cannot process more than that many tokens a minute, so it cannot spend faster than that, ever. Requests beyond it are throttled, not billed.

WHY IT IS THE REAL CONTROL

Budgets only email you

A spending alert tells you after the fact, up to a day late. The TPM cap stops the spend as it happens. It is the difference between a smoke alarm and a fire door.

THE FAILURE FROM LECTURE 1, PRICED

Remember the agent that looped forever because we removed its budget. On a real deployment, that loop spends real money for as long as it runs. The TPM cap is the same idea as your `max_steps`, one level down, protecting you from the loop you forgot to bound. Set it low. You can raise it in two clicks. You cannot un-spend money.

Getting it into your code

```
# .env
PROVIDER=foundry
AZURE_OPENAI_ENDPOINT=https://cse476-foundry.openai
                        .azure.com/openai/v1/
AZURE_OPENAI_API_KEY=your-key-here
MODEL=chat-demo        # the DEPLOYMENT name
```

Four lines. Then PROVIDER=foundry, and every notebook in the course runs on this deployment instead of the free lane. Nothing else in any notebook changes. That is the whole point of the lane abstraction from Lecture Zero.

AND THE KEY IS A SECRET

The endpoint is fine to share. The key is not. Never screenshot it, never paste it in a cell whose output gets committed. If it ever leaks, regenerate it immediately, which takes ten seconds in the portal. You will practise this reflex all semester.

Hold on to this

Foundry is the industrial version of the machine you already built by hand.

Projects, not hubs

And check the New Foundry toggle is on.

A deployment is not a model

MODEL in your code is the deployment name.

Check the lifecycle before you build

Deprecated works today and dies on a date.

The TPM cap is the real cost control

Budgets only email you, and always too late.

Before next session

OPTIONAL

Stand up your own Foundry

Only if you want to. The README has the full walkthrough, including the free trial and the cost cap. Everything below still runs free on the GitHub lane.

DO

Find the deployment name in your code

Open any notebook. Find where MODEL is read. Write one sentence on why it is the deployment name and not the model name.

THINK

Price a runaway

Your agent loops, uncapped, for one hour at your TPM limit. Roughly how many tokens is that? You now have the number to reason about a real bill.

ON THE OPTIONAL ONE

I demonstrate on my account so nobody has to spend a rupee. If you set up your own, do it because you want the experience of the real platform, not because the course requires it. It does not.

End of Unit 1

Unit 2, Building Intelligent Agent Workflows

You can now build an agent, choose its architecture, keep it affordable, make it reason, design it well, and reach it through the enterprise platform. Unit 2 asks the harder question underneath all of it: when should this even be an agent, and when is a plain workflow the better, cheaper, more reliable answer. That decision is where good engineers earn their keep.

COME WITH

Practical 2 finished if it is not already, and the two short answers from the previous slide.